2025 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER) | 979-8-3315-3898-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/DISCOVER66922.2025.11259028

Two-Tier Intrusion Detection Mechanism in SOME/IP Automotive Communication Systems

Bhargav Jammalamadaka
Department of Electronics and Communication Engineering
Amrita School of Engineering, Coimbatore
Amrita Vishwa Vidyapeetham, India
Email: cb.en.p2ael23024@cb.students.amrita.edu

Gandhiraj R
Department of Electronics and Communication Engineering
Amrita School of Engineering, Coimbatore
Amrita Vishwa Vidyapeetham, India
Email: r_gandhiraj@cb.amrita.edu

Manoj Kumar Panda
Department of Electronics and Communication Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
Email: kp_manoj@blr.amrita.edu

Arpit Agarkar
Centre of Excellence ADAS,
Tata Consultancy Services,
Pune, India.
Email: arpit.agarkar@tcs.com

***Abstract*—Automotive Ethernet, the primary backbone network for in-vehicle connectivity, is replacing conventional Controller Area Networks (CAN) as the automotive industry transitions to Software-Defined Vehicles (SDVs). Thus, in line with the adaptable and dynamic architecture of SDVs, the Scalable Service-Oriented Middleware over IP (SOME/IP) protocol has emerged as a crucial facilitator of service-oriented communication within in-vehicle networks (IVNs). However, because of a rise in external attack vectors and intrinsic protocol flaws, integrating SOME/IP into SDV ecosystem, exposes these networks to serious security threats. Despite its significance, not much study has been done on safeguarding SOME/IP-based IVNs from possible security breaches. In order to solve these issues, this study applies both rule-based and deep learning (DL)–based anomaly detection algorithms to improve SOME/IP security. While DL-based models enable adaptive and intelligent threat identification by learning from network data patterns, rule-based techniques give deterministic threat detection. This study intends to efficiently detect and reduce security risks by utilising these complementary techniques, enhancing the general robustness and security of SDV platforms and service-oriented automotive networks.**

***Index Terms*—Automotive Ethernet, SOME/IP, in-vehicle networks**

I. Introduction

The emergence of Software-Defined Vehicles (SDVs), in which software increasingly governs a vehicle's operations to improve user experience and operational efficiency, is causing a major upheaval in the automobile industry, because this change necessitates sophisticated communication systems, automotive Ethernet—which offers greater capacity and scalability replaces conventional Controller Area Networks (CAN) [1]. Service oriented architectures (SOA) in in-vehicle networks (IVNs) are made possible in large part by the Scalable Service-Oriented Middleware over IP (SOME/IP) protocol [2]. However, there are serious security flaws introduced when SOME/IP is integrated into SDVs. The intrinsic flaws in the protocol are more noticeable when external attack avenues increase [3]. Research on protecting SOME/IP from new threats is scarce, despite its critical importance in contemporary IVNs. The high-bandwidth and dynamic nature of SOME/IP networks is not adequately addressed by current security solutions, which are mainly made for CAN-based systems [4]. By using both rule-based and deep learning (DL)-based anomaly detection algorithms, this study aims to improve SOME/IP security. While rule-based methods make it quick in identifying the deterministic anomaly [5], DL algorithms can learn from patterns to identify novel risks [6].

The paper is structured as follows: Section II reviews existing literature on various methods that were implemented previously by the scholars, and highlights the contributions of this paper relative to prior work. Section III details the SOME/IP and vulnerabilities. Section IV details the proposed methodology and system architecture. Section V describes the experimental setup. Section VI presents the results and their analysis. Finally, Section VII concludes the paper. The contributions in this paper are:

- *SOME/IP Data Generation Framework*, We suggest a process to create realistic SOME/IP data with CarMaker and the VSOMEIP stack to test and train security models for in-car communication simulation.
- *A two-layer IDS design is proposed*, The architecture integrates a defence-in-depth strategy, where rule-based detection rapidly analyses header data for known threats and deep learning-based payload inspection to identify advanced and unknown threats.
- *Behaviour modelling to detect stealth attack*, The system uses rate-of-change analysis to identify changes in communication behaviours to improve the detection of event-driven messages like first-time or stealth attacks, which are hard to detect with traditional measures.
- *Even spoof prevention*, We propose a Global Session ID as a secure token mechanism to uniquely identify and trace ECU-to-ECU communication sessions. This

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

aids in message authenticity and prevents spoofing or session hijacking attacks. We employ TensorRT for model optimisation and inference speed-up to fulfil real-time performance requirements, with significantly lower latency than regular Python implementations.

The suggested IDS system has been implemented and tested in a Driver-in-the-Loop (DIL) simulation environment on a High Performance Computing (HPC) platform integrated into a test setup. This offers a real automotive environment to test the system's performance and reliability in real time.

II. Literature survey

The increasing use of Ethernet in vehicles has led to the adoption of Scalable Service Oriented Middleware over IP (SOME/IP) for in-vehicle network (IVN) communications. However, SOME/IP lacks inherent security features, making it vulnerable to various cyberattacks such as Denial-of-Service (DoS), masquerade, and replay attacks. Among these identified vulnerabilities, various security methods have been approached, but [7] Intrusion detection systems and cryptographic mechanisms attracted major focus[8]. The most promising is a hybrid approach: this combination of rule-based and deep learning-based IDS outperformed other solutions on detection performance both for previously known and unknown attacks[9]. Rule-based IDS systems apply real-time monitoring efficiently through protocol violations and anomalies in message headers, intervals, and communication behaviours [10]. In a research paper, they proposed a multi-layer IDS in which the rule based module checks SOME/IP headers and message intervals[11], and the DL-based module uses some sophisticated algorithms for the detection of payload anomalies[12]. This hybrid IDS has achieved high accuracy both on DL-based and rule-based detection with real-time performance, very suitable for in-vehicle applications. On similar grounds, constructed a machine learning-based IDS was constructed with the RF and RNN models[13]. It was efficient in detecting DoS, MFA attacks, and replay attacks by analysing packet headers and timestamps. It outperformed traditional IDS in the detection of sophisticated attack patterns [14].

Yet, while the rule-based systems deter the known threats quite effectively, their power declines against new or evolving threats[15]. Finally, the systems based on DL are more flexible because they learn from the data how to discover new attack patterns[16]. This combination ensures comprehensive protection: The rule based module can address protocol violations in a fast manner, while the DL-based module addresses complex attack scenarios. Eavesdropping, one of most ignored type of attack, which does not effect the system but is harmful when seen from prospective of privacy [17]. Furthermore, proposed an authentication and secure communication scheme to address the lack of encryption and authentication vulnerabilities in SOME/IP[18].

Their scheme is based on a symmetric key, managed through a centralised KMC that is periodically updated to make the scheme resistant to brute force attacks. They further modified the SOME/IP data frame using the AEAD algorithm, which provides a message. One of the most used tools for simulating vehicle dynamics with CAN or other kinds of communication is CarMaker.This provides the platform to collect,analyze and visualise the data, using which we can make a control loop[19] to control the vehicle.The different approaches towards solving this are summarised in "Table. I"

TABLE I
Comparison of Security Techniques

Technique	Category	Algorithm Definition	Merits
Authentication Ticket	Access Control	cryptographic tokens issued by a trusted authority.	Secure, reliable reduces password re-entry.
Rule-based System	Intrusion Detection	Uses predefined rules to detect malicious behavior.	Fast detection, low resource usage.
Deep Learning System	Anomaly Detection	Uses RNN/CNN to learn behaviour for real-time detection.	Detects attacks, adapts for the evolving threats.

III. SOME/IP

SOME/IP is located above the OSI model's fourth layer and is based on the TCP/UDP protocol. Establishing a common middleware for IP-based communication within the car is its goal. One of the essential elements for implementing in-car network connection under the service-based architecture is SOME/IP. We begin by outlining the SOME/IP communication method and elucidating the use cases for Event and Remote Procedure Call (RPC) packets. Next, we examine the types and situations of attacks against the SOME/IP protocol.

A. Overview of SOME/IP

Scalable service-oriented middleware over IP, or SOME/IP, is a communication protocol designed to facilitate effective and adaptable data sharing in in-car networks. With the help of its implementation of a service-oriented architecture (SOA), many Electronic Control Units (ECUs) can connect with one another over Ethernet using IP-based messaging. The demand for greater bandwidth and scalability in contemporary automobiles, especially as they get outfitted with cutting-edge technologies like infotainment, sensor fusion, and autonomous driving, is what pushing this shift towards Ethernet-based communication.

Through the availability of essential capabilities like message serialisation, event alerts, and service discovery, SOME/IP enables ECUs to dynamically find and connect with one another without the need for pre-configured settings. In

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

complicated automotive applications where several ECUs must share data in real-time, such as infotainment, diagnostics, and vehicle management, this flexibility is crucial. Despite these benefits, SOME/IP makes it harder to guarantee dependable and secure communication because it is more vulnerable to IP-based attacks. To handle possible threats in this increasingly interconnected automobile environment, sophisticated security measures and real-time monitoring are required.

B. Vulnerability of SOME/IP

- Fuzzy This refers to the part of a SOME/IP packet that contains metadata about events or RPCs, such as event IDs, request IDs, and payload length. Fuzzing this header means manipulating these fields with random or out-of-bounds values
- Spoof is seen to be an improvement over fuzzy. According to our definition, the payload is not contained in the fuzzy targets. If the SOME/IP header does not meet the requirements, the fuzzy on the payload is deemed invalid. Spoofing allows the attacker to alter or replay the event's content while simultaneously sending the SOME/IP header as needed. This calls for a greater degree of communication system expertise.
- Denial of service (DoS) occurs when an attacker alters the cycle of periodic events or SOME/IP packets to clog the network. DoS can also be accomplished by flooding the network with traffic that has nothing to do with SOME/IP.

C. VSOME/IP

VSOMEIP, as an implementation of the SOME/IP protocol, is designed to provide a standardised way for ECUs (Electronic Control Units) within a vehicle to communicate over an IP-based network. SOME/IP plays a crucial role in automotive systems by enabling a service-oriented architecture that facilitates communication across diverse systems, like infotainment, autonomous driving features, and vehicle diagnostics.

VSOMEIP adheres to AUTOSAR standards by implementing the SOME/IP (Scalable service-Oriented Middleware over IP) protocol, which is the standard middleware communication protocol in AUTOSAR-based systems. It supports service-oriented communication, allowing different Electronic Control Units (ECUs) within the vehicle to discover and use services dynamically, aligning with AUTOSAR's architecture for scalable and flexible communication. By managing service discovery, message serialisation/deserialization, and communication over IP networks (using TCP/UDP), VSOMEIP ensures reliable and efficient message exchange between ECUs. Furthermore, VSOMEIP is fully integrated into the AUTOSAR communication stack, acting as a middleware layer that adheres to AUTOSAR's requirements for transport, data exchange, and service management.

The open-source nature of VSOMEIP, supported by contributions from COVESA, ensures interoperability across vendors, making it a reliable and close approximation to real-world in-vehicle communication systems. The service discovery protocol implemented in VSOMEIP complies with AUTOSAR's dynamic service discovery, enabling ECUs to discover and interact with services at runtime. Additionally, VSOMEIP's modular and scalable design fits AUTOSAR's principles, allowing for easy integration of services and clients in complex vehicle systems while efficiently handling resource and timing constraints critical for automotive applications. By providing a robust and flexible implementation of the SOME/IP protocol, VSOMEIP closely aligns with AUTOSAR's real time communication standards and industry requirements.

IV. Proposed Methodology

A. Data Generation

There is no prominent dataset in the industry for SOME/IP. Since SOME/IP communication is not much in use in current vehicles in the market, which are mass-produced. In previous academicians have developed SOME/IP data generators, but they have only the header and a dummy payload. The dataset available in the public domain contains a very small number of samples, which are not helpful while training a Deep learning model.

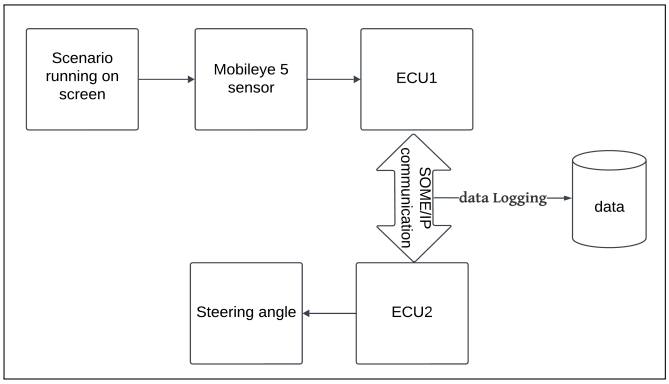

Fig. 1. Data generation method using a real-time Driver in loop system

A Driver in a loop system is used "Fig. 1" in order to generate the data using tool-chain of CarMaker, VSOME/IP. The ECU1 is connected to a MobilEye sensor, which is mounted on top of the windshield then a scenario is run on the screen, which will look like the vehicle is moving. Based on the vehicle on the screen and the Lane markings shown, Mobileye5 will give the obtained lane value and object distance value. Which then is provided as a service when ECU 2 request the sensor data in order to control the steering. A publish-subscribe mechanism is established between the service and client ECU's in the VSOMEIP.

B. Attack injection

The types of attack that are taken into this research are given below:

- Replay Attack: In a replay attack, the header row of the former sample is simply copied over as it is. The payload data however is taken from another set of data so that even if the structure is the same, the content could be different.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

This process essentially replicates an earlier sent message so that it would appear to be resent over the network.

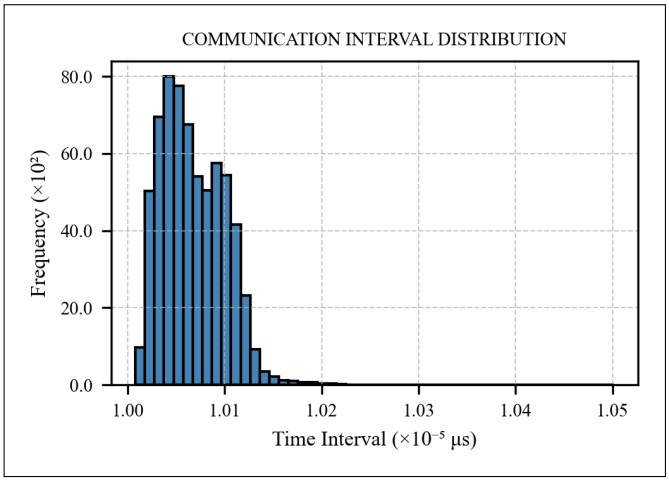

Fig. 2. Time interval between two successive communications

- Denial-of-Service (DoS) Attack: By examining the timestamps, it has been seen that even though the cycle time is specified as 100 milliseconds"Fig. 2", there is an inherent delay from a minimum of 79 microseconds to a maximum of 5.3 milliseconds.
  One of the most typical features of the DoS attack is that the difference in the timestamp is always less than the anticipated 100-millisecond time difference. This is a sign of an abnormal rise in the rate of sending the message, flooding the system and in the process, possibly degrading its performance.
- IP Spoofing: In an IP spoofing attack, the header is identical to that of legitimate nascent data, having the same anticipated format. The payload is corrupted or malicious data that is meant to deceive the system. The rate of change in the critical parameters also exceeds the limits specified in the driver profile, and this leads to inconsistencies that can be exploited to identify unauthorised or fraudulent activity.

C. Dataset

The data log during the communication of the ECU's has been logged as a text file. The obtained text file is then converted into csv file for ease of use in future for the Deep learning model. The obtained number of sample are 2,68,837 for the communication scenario. This is used as a base to actually design the rule based system, as we have the header of the SOMEIP. In comparison to the publicly available datasets used in previous research, our system is built upon a much larger and more comprehensive set of samples, which greatly enhances its effectiveness and scope in detecting anomalies or attacks. We created 8 different attack instances and recorded the data, each with 2,68,837 samples. The increased volume and diversity of the data allow us to better capture a wide range of scenarios and potential threats, improving the accuracy and robustness of our intrusion detection system. This larger dataset equips the system with the ability to generalise better, covering a broader spectrum of normal and abnormal network traffic patterns that might occur in a real-world in-vehicle network.

D. Anomaly Detection Framework and Workflow

The system begins with a dedicated training phase, where a dataset of network messages is pre-processed, including feature extraction and normalisation. This prepared data is used to train the deep learning model to recognise patterns of normal and anomalous behaviour. Once the DL model is sufficiently trained, it is deployed into a real-time environment to work alongside the rule-based system. In the real-time detection phase, incoming SOME/IP messages are processed through a data extraction module to retrieve key information for analysis.

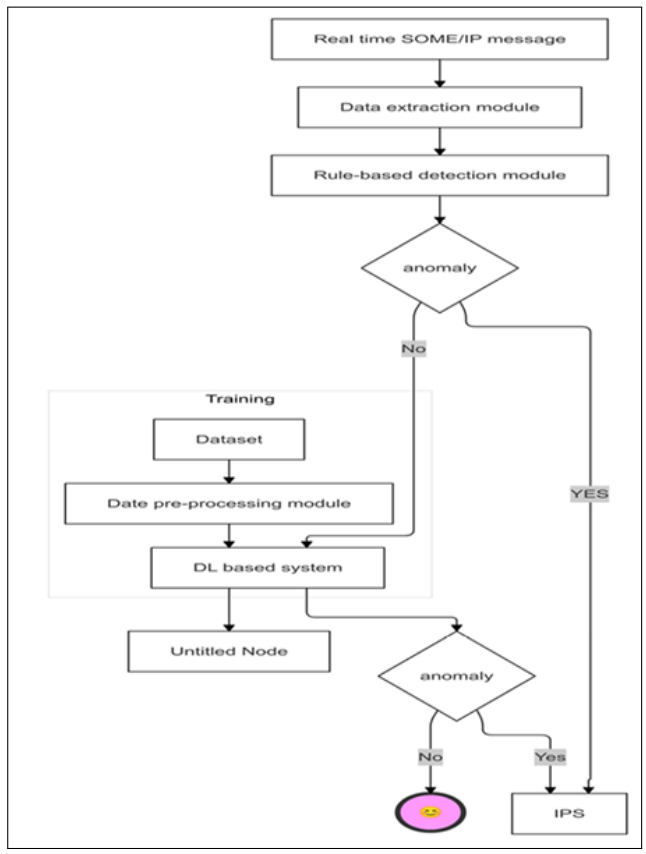

Fig. 3. Workflow and architecture of IDS

This extracted data is first analysed by the rule-based detection module, which analyses using predefined rules based on known threats. If an anomaly is detected, the system triggers the Intrusion Prevention System (IPS), which may initiate corrective actions such as blocking or logging the threat. If no anomaly is found, the data are sent to the trained DL system, which has the capability to look into more complex patterns that are subtle as shown in"Fig. 3".

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

E. Rule-based System

Inspecting single packets in isolation can determine whether a message is well formed. In order to analyse a whole conversation between two devices, information from multiple packets is needed. As the rules are evaluated every time a packet arrives, the system may react in a timely manner[14].

The following details are known in advance:

- Every server's MAC and IP addresses
- Each server's expected message types, service and method IDs, and supplied services
- Each client's MAC address, IP address, and client ID
- The methods and services that a client can access on a server

the rule-based security mechanism"Fig. 4" is activated immediately after the client or service receives a message in the VSOMEIP environment. Once the message is received, VSOMEIP handles the deserialization of the serialised data, converting it into a usable format. The next critical step involves extracting both the header and payload from the de-serialised message. At this point, the header—containing crucial metadata about the message, such as the source, destination, and type of message—is subjected to a set of predefined rules that you have integrated directly into the C++ files of the client or service application. These rules are especially designed to check the integrity and authenticity of the message header to see if it complies with certain security policies before further processing is done. This could be everything from checking for valid message IDs and source addresses of acceptable types of messages, dependent on what fits into your security policies. The purpose of this step is to quickly filter out malicious or malformed messages that could pose a threat to the system. If the message header is valid according to all these rule-based checks, it is assumed to be a valid message, and its payload data is allowed further into the next stage. A more sophisticated deep learning-based Intrusion Detection System analyses the payload for hidden patterns of malicious activity or anomalies.

It provides speed and depth-security because it is a two-layered approach, encapsulating the accuracy of not only fast, superficial screening in the rule-based system but also the nuanced detection in a deep learning model of complex attacks. In case the header does not meet the required conditions laid out in the rule set—including incorrect format of the message, suspicious sender information, or any other clue showing this is an abnormal message—the system immediately flags it as an attack. Anyway, the payload is not further processed, and the system may log this event or react by performing predefined actions, such as alerting an administrator or blocking future messages coming from this source. This approach ensures that attacks are intercepted as early as possible, minimising the risk of them reaching the deep learning-based IDS or affecting critical vehicle systems.

The rule based section comes into the picture after the client or the service receives the message. The serialised data is de-serialised by the VSOMEIP, and the data extraction part come

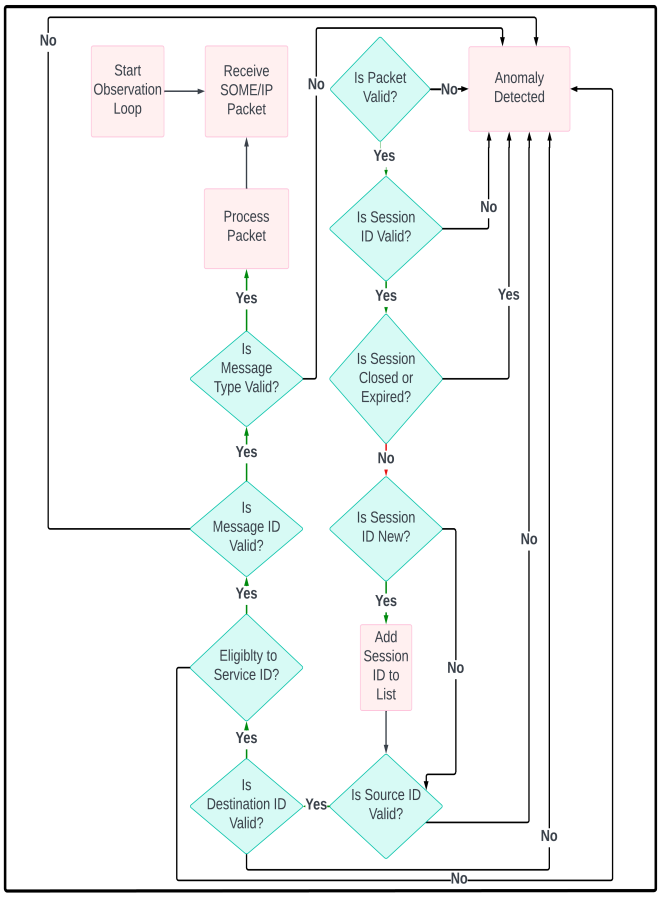

Fig. 4. Rule-Based system flow

into the picture immediately after the data is extracted. We will subject the obtained header to the rules that we have integrated into the cpp files of the client or service. When the header successfully passes through all, then the payload data is allowed to proceed further to the DL-based IDS system. If the header doesn't comply with the set of rules, then the message is considered to be an attack.

F. DL Based Detection System

a) Pre-processing Modules: The first step in training the LSTM model is to collect, organise, and pre-process the raw SOME/IP network traffic data for analysis. Since there are multiple CSV files of network traffic samples, the data must be loaded and concatenated into a single dataset efficiently. Before feature extraction, the dataset is scanned for missing or inconsistent values. Missing values are either removed or imputed, if any, to prevent errors at later stages. Duplicate records are also scanned and removed to maintain dataset integrity. The relevant features to be used in intrusion detection, such as VC.Brake, VC.Gas, Vhcl.Steer.Ang, and Vhcl.sRoad are selected. The target variable is assigned as the label column of 0 for normal traffic and 1 for attacked traffic. In this way, the model learns from normal traffic and abnormal activity in the dataset.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

One of the most important transformations is computing the rate of change of key features. Since network anomalies often manifest as sudden deviations, the rate of change helps in detecting such irregularities. For each feature X, the rate of change is calculated as:

$$\Delta X_t = X_t - X_{t-1} \tag{1}$$

where X_t is the current value and X_{t-1} is the previous value.

b) Model Architecture: The Forget Gate controls what information from the past to retain or forget. It controls whether information from the previous time step is to be retained or forgotten. The decision is made on the basis of the last hidden state, the current input, and a learnable weight and bias matrix. A sigmoid function is employed so that the output is restricted to between 0 and 1, with output values of approximately 1 retaining information and output values of near 0 will forget it. It is computed using the equation:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{2}$$

where f_t is the forget gate activation, W_f and b_f are the weight matrix and bias, h_{t-1} is the previous hidden state, x_t is the current input, and σ represents the sigmoid activation function.

The Input Gate selects the new input to be stored in the cell state. It has two components: an activation function that selects the importance of the new input and a candidate cell state that is computed before it is stored. The activation function determines the size of the new input to be stored, and the candidate cell state is the computed input data. They are both added to efficiently update the cell state. It consists of two equations:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{3}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{4}$$

where i_t is the activation controlling new information, $\tilde{C}_t$ is the candidate cell state, and W_i, W_C, b_i, b_C are the corresponding weight matrices and biases.

Cell State Update blends Forget and Input Gate outputs to maintain long-term dependencies. The previous cell state is weighted by the Forget Gate output to retain or forget the previous information. Meanwhile, the candidate cell state, regulated by the Input Gate, adds new relevant information to the updated cell state. Thus, strong patterns of the previous sequences are retained, and new information is appended. Equation integrates both the forget and input gate outputs to update the long-term memory:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{5}$$

where C_t is the updated cell state and $\odot$ denotes element-wise multiplication.

The Output Gate controls what to pass on to the next time step or final output from the updated cell state. It takes the previous hidden and input and passes them through a learnable weight matrix and bias, capped with an activation function. The filtered output is multiplied by the transformed cell state to produce the final hidden state, and it's sent on to the subsequent LSTM cell or used in the decision-making of the network. Output Gate determines what information should be passed to the next LSTM layer or output. It is computed as:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{6}$$

$$h_t = o_t \odot \tanh(C_t) \tag{7}$$

where o_t is the output gate activation and h_t is the hidden state output.

Following the LSTM layer, a **fully connected Dense layer** is added to refine feature representations. This layer consists of 50 neurons and uses the **ReLU activation function**, which introduces non-linearity and helps the model learn complex relationships in the extracted features. The Dense layer is computed as:

$$h_{\text{dense}} = \text{ReLU}(W_d \cdot h_{\text{lstm}} + b_d) \tag{8}$$

where:

- h_{dense} is the output of the Dense layer,
- W_d and b_d are the weight matrix and bias for the Dense layer,
- h_{lstm} is the output of the LSTM layer.

The final layer is the **Output Layer**, which consists of a single neuron and applies a **Sigmoid activation function**. Since the model performs **binary classification** (Normal vs. Attack), the Sigmoid function ensures that the output is a probability score between 0 and 1:

$$\hat{y} = \sigma(W_o \cdot h_{\text{dense}} + b_o) \tag{9}$$

where:

- $\hat{y}$ is the predicted probability of an attack.
- W_o and b_o are the weight matrix and bias for the output layer.

The model classifies network traffic based on the threshold:

$$\hat{y} \geq 0.5 \Rightarrow \text{Attack (1)}, \quad \hat{y} < 0.5 \Rightarrow \text{Normal (0)} \tag{10}$$

This final layer enables the model to make accurate intrusion detection decisions, leveraging the feature representations learned from the previous layers.

V. Experiment Setup

My setup has Intel(R) Core(TM) i7-9750H CPU clocked at 2.60GHz and 16GB of RAM.using Ubuntu 20.4.6 OS.This setup has also seen throughput confirmation of VSOMEIP, and the IDS has been implemented on this setup too. We perform real-time vehicle-level analysis of the proposed Comprehensive IDS on this setup, considering the detection time of both rule-based and accuracy AI-based modules by CPU.

To test the real world performance, it is kept in a Driver in loop system which has a HPC(Nuvo-8108GC, Intel® 9th Gen Core i7 LGA1151, NVIDIA® RTX A6000) connected to steering and throttle control systems which act as domain(6-core Arm® Cortex-A78AE, NVIDIA Ampere GPU) controllers this will represent a real-time environment of a vehicle.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

VI. Results

A. Data description

The dataset generated by the method mentioned in the data generation module uses a driver-in-loop system with a Mobileye camera and lane keep assist system, in which the steering system is in request-response type of communication, and the throttle and brake control system are in event based communication with the perception module to observe any obstacles. We have 8 different instances with each 2,68,837 samples collected.

In the dataset above, the experiments validate that the rule-based detection proposed here has a 100 percent detection rate. This is expected because rule-based decisions are accurate. Real-time performance is a critical measure for rule-based detection.

In this paper, the rules are implemented in program code in the form of logical judgments and not in a rule base. This is a more suitable arrangement in an embedded environment because it will not occupy extra memory of a rule base or compromise real-time performance due to rule retrieval. Positive logical judgments (in whitelist mode) mark packets that do not satisfy the requirements as abnormal directly.

To create a training set that has 80 percent of the data samples and a test set that contains 20 per cent of the data samples, we employ an 80–20 percent train-test split. Until the last hold-out validation, the test set won't change.

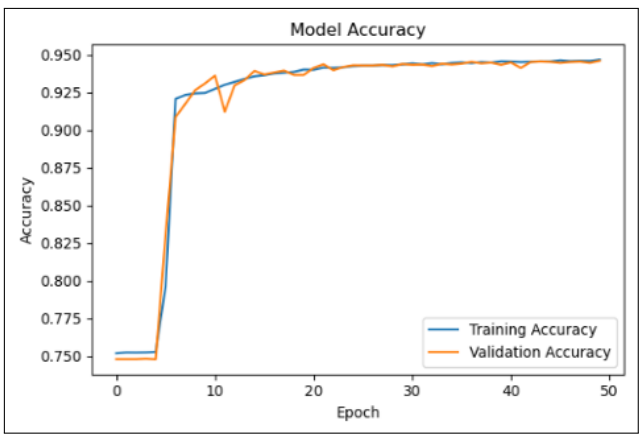

Fig. 5. training accuracy of the model

It should be noted that the header and replay, tamper attack cycle satisfies the system requirements. The attackers could be said to have successfully impersonated the odes that contain these services and spoofed the rule-based IDS. The payload would therefore be the target of these attacks.

B. Performance Metrics

Performance evaluation of the model is carried out using various metrics, including accuracy and recall.

1) Accuracy: Accuracy is the ratio of correctly predicted instances to the total instances and is given by:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

The models accuracy can be observed in the "Fig. 5".

2) Recall: Recall, also known as Sensitivity or True Positive Rate, measures the ability of the model to identify positive instances correctly. It is defined as:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{12}$$

TABLE II
Model Performance Metrics

Metric	Value
Accuracy	94.583%
Precision	93.29%
Recall	99.43%
F1-Score	96.30%

The value obtained from the testing when the model is subjected to 13,109 samples is illustrated in the form of a Confusion matrix "Fig. 6" and the performance metrics are tabulated in "TABLE. II".

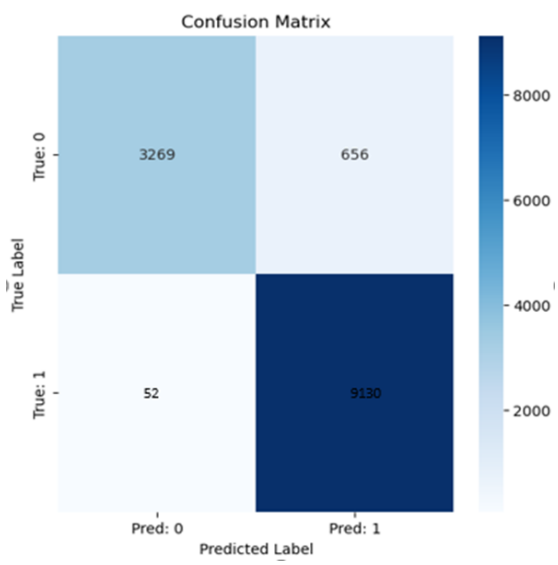

Fig. 6. confusion matrix of model testing

The total SOME/IP communication time is 452 microseconds with 80 microseconds of overhead from the rule-based system.

Though the system accurately identifies SOME/IP abnormalities, the rule-based system cannot identify IP spoofing and Denial-of-Service (DoS) attacks. Such higher-level attacks are effectively identified by the deep learning-based solution able to identify traffic behaviours beyond the basic static rule examinations. "TABLE. III"summarizes the performance comparison between the Laptop and Nuvo-8108CC setups.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.

The Nuvo-8108CC demonstrates significantly lower communication intervals and processing latencies in both the rule-based and deep learning-based systems, indicating its suitability for real-time embedded applications. In contrast, the Laptop, equipped with a high-performance NVIDIA RTX A6000 GPU, supports the H5 model, which allows for more flexible deep learning experiments but results in higher latency.

TABLE III
COMPARISON OF EXPERIMENTAL SETUPS.

Experiment Setup	**Laptop**	**Nuvo-8108GC**
CPU Configuration	Intel® Core i7-9750H	Intel® 9th Gen Core i7 LGA1151
GPU	-	NVIDIA® RTX A6000
Single Communication	100ms	30ms
Rule-based System	800µs	600µs
DL-based System	2.4ms	940µs
Model Type	H5	ONNX

VII. CONCLUSION

This research suggests a security model for SOME/IP communication in vehicle networks by incorporating a rule-based intrusion detection system (IDS) and a deep learning-based anomaly detection model. The system was able to detect Denial-of-Service (DoS) attacks, IP spoofing, and replay attacks effectively.

Performance measurement indicated that detection accuracy as a whole is 94.583. Detection time is 100 microseconds on average, with 80 microseconds of overhead of the rule-based system. While the rule-based method detects SOME/IP anomalies well, it cannot detect sophisticated attacks such as IP spoofing and DoS. The latter can be efficiently dealt with by the deep learning model.

The integration of deep learning improves the detection feature, offering a multi-layered automotive network security solution. Future research can be directed towards simplifying the system for real-time applications and extending its application to detect future cyber threats in connected vehicle ecosystems.

REFERENCES

[1] M. D. Natale, H. Zeng, P. Giusto and A. Ghosal, 'Understanding and Using the Controller Area Network Communication Protocol: Theory and Practice', New York, NY, USA:Springer, 2012.

[2] P. Wolf, R. Pöschmann, and J. Dittmann, "Security Challenges of SOME/IP for Automotive Service-Oriented Architectures," Proc. IEEE Int. Conf. on Vehicle Electron. and Safety, 2017, pp. 32-37.

[3] T. Hoppe, S. Kiltz, and J. Dittmann, "Security Threats to Automotive CAN Networks – Practical Examples and Selected Short-Term Countermeasures," Proc. Int. Conf. on Comp. Safety, Reliability, and Security (SAFECOMP), 2008, pp. 235-248.

[4] B. Manimekala, K. S. Divya and S. Chakrabarti, "A Study On Secured Data Transmission, Key Management And Its Comparative Analysis Of Scheduling Techniques In VANET," International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, 2021, pp. 283-288.

[5] A. Kavousi-Fard, M. Dabbaghjamanesh, T. Jin, W. Su and M. Roustaei, "An Evolutionary Deep Learning-Based Anomaly Detection Model for Securing Vehicles," in IEEE Transactions on Intelligent Transportation Systems, vol. 22, no. 7, pp. 4478-4486, 2021.

[6] J. Sun et al., "A Tamper-Resistant Broadcasting Scheme for Secure Communication in Internet of Autonomous Vehicles," IEEE Transactions on Intelligent Transportation Systems, vol. 25, no. 3, pp. 2837-2846, 2024.

[7] F. Luo, Z. Yang, Z. Zhang, Z. Wang, B. Wang, and M. Wu, "A Multi-Layer Intrusion Detection System for SOME/IP-Based In-Vehicle Network," Sensors, vol. 23, no. 4376, April 2023.

[8] J. Heo, H. Kim, and H. J. Jo, "SOME/IP Intrusion Detection System Using Machine Learning," IEICE Transactions on Information and Systems, vol.E105-D, no. 11, pp. 1923-1931, 2022.

[9] B. Ma, S. Yang, Z. Zuo, B. Zou, Y. Cao, X. Yan, S. Zhou, and J. Li, "An Authentication and Secure Communication Scheme for In-Vehicle Networks Based on SOME/IP," Sensors, vol. 22, no. 647, Jan 2022.

[10] S. B. Oh, J. H. Kim, S. W. Lee, J. B. Park and J. W. Jeon, "Implementation of GUI-based SOME/IP and SOME/IP-SD communication," 2023 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia), Busan, Republic of Korea, 2023, pp. 1-4.

[11] K. Iehira and H. Inoue, "Feasibility Assessment of Denial-of-Service Attacks by Analyzing SOME/IP-SD State Transition Models," Computers Software and Applications Conference (COMPSAC), Osaka, Japan, 2024, pp. 1732-1738.

[12] M. Vujanić, N. Trifunović, I. Kaštelan and B. Kovačević, "Bitroute SOME/IP: Implementation of a Scalable and Service Oriented Communication Middleware," International Convention on Information, Communication and Electronic Technology (MIPRO), Opatija, Croatia, pp. 1426-1429, 2022.

[13] K. Wang, Y. Guo, W. Zhao, Q. Zhou, and P. Guo, "Gas path fault detection and isolation for aero-engine based on LSTM-DAE approach under multiple-model architecture," Measurement, vol. 202, pp. 111875, Sept 2022.

[14] N. Herold, S. -A. Posselt, O. Hanka and G. Carle, "Anomaly detection for SOME/IP using complex event processing," NOMS 2016 - 2016 IEEE/IFIP Network Operations and Management Symposium, Istanbul, Turkey, pp. 1221-1226, 2016.

[15] Y. Hwang, C. Choi, R. W. Wardhani, D. S. C. Putranto, and H. Kim, "Enhancing Structured Query Language Injection Detection with Trustworthy Ensemble Learning and Boosting Models Using Local Explanation Techniques," Electronics, vol. 13, no. 22, pp. 4350, 2024.

[16] N. Kabilan, Vinaykumar Ravi, and V. Sowmya, "Unsupervised intrusion detection system for in-vehicle communication networks," Journal of Safety Science and Resilience, vol. 5, no. 2, pp. 119–129, June 2024.

[17] G. Vieeralingaam, R. Ramanathan, and T. Ohtsuki, "Resource Allocation for Secure MIMO-SWIPT Systems in the Presence of Multi-Antenna Eavesdropper in Vehicular Networks," Sensors, vol. 23, no. 19, Art. no. 8069, Sept 2023.

[18] G. Kirubavathi, I. R. Sumathi, J. Mahalakshmi, and D. Srivastava, "Detection and mitigation of TCP-based DDoS attacks in cloud environments using a self-attention and intersample attention transformer model," The Journal of Supercomputing, vol. 81, no. 3, Art. no. 474, Feb 2025.

[19] A. S. Agarkar, R. Gandhiraj and M. K. Panda, "Driver Drowsiness Detection and Warning using Facial Features and Hand Gestures," International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN), Vellore, India, 2023, pp. 1-6.

Authorized licensed use limited to: Pontificia Universidade Catolica do Rio Grande do Sul (PUC/RS). Downloaded on March 26,2026 at 05:04:13 UTC from IEEE Xplore. Restrictions apply.